

Feature Detection & Description

Prof. Jungong Han (jungong.han@sheffield.ac.uk)
Department of Computer Science
The University of Sheffield

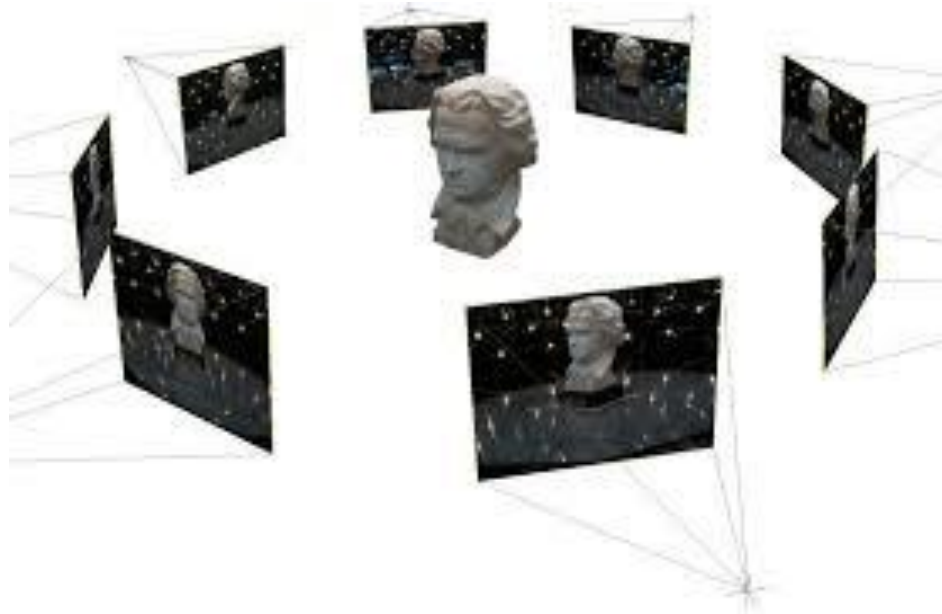
Puzzles

- Regarding image warping, which statements are **false**?
 - Forward warping may result in missing information in the warped image in case transformed coordinates "fall" on non-integer locations
 - Backward warping requires interpolation of the intensity from the intensities neighbouring the transformed coordinate
 - Bi-linear interpolation produces better results than bi-cubic interpolation
- What is an homography?
 - It's a transformation that maps a plane to another plane
 - It's an Affine transform
 - It's a rotation transform

Outline

- Features/keypoints/interest points
- Keypoints detectors
- Keypoints descriptors
- Matching

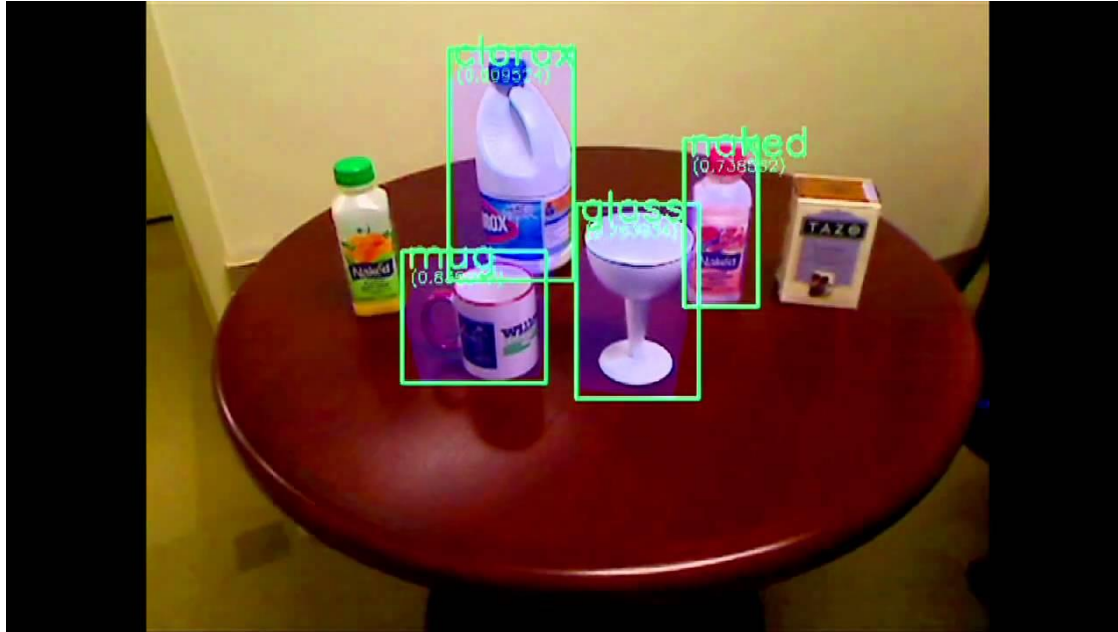
Motivation



3D Reconstruction

Credit: Saikia

Motivation

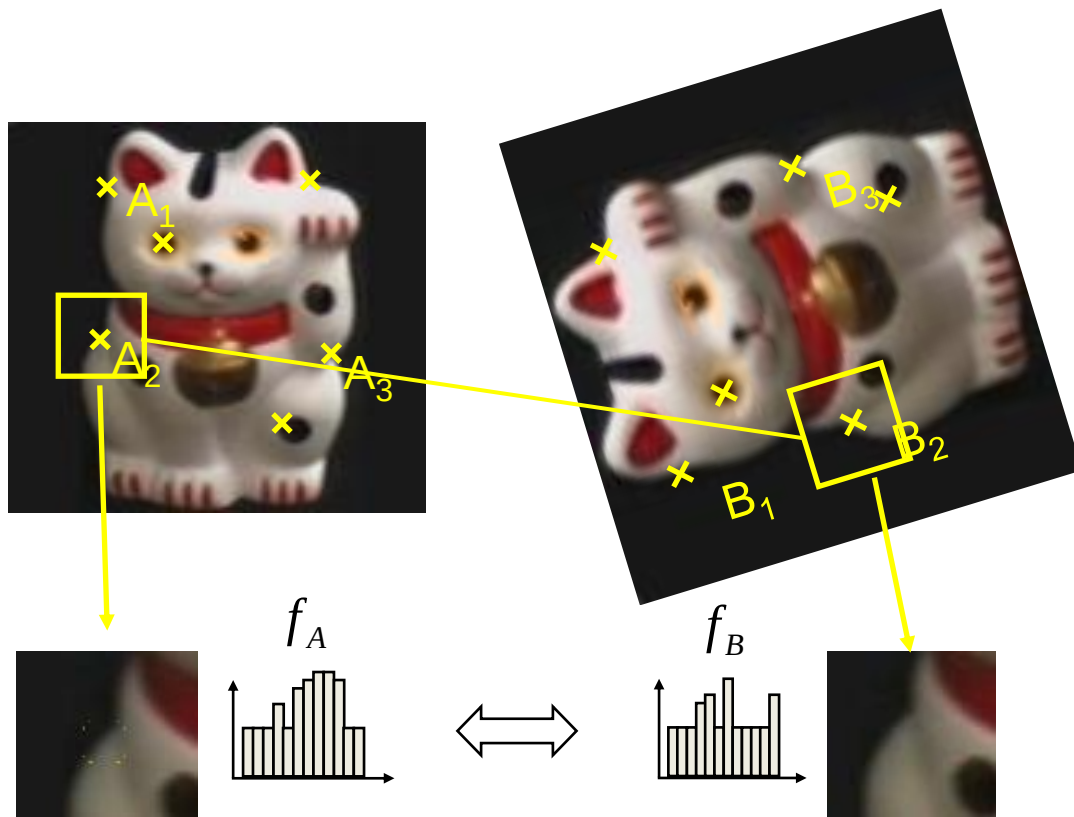


Object Detection and Recognition

(www.youtube.com/watch?v=tIC2O9T9jks)

Many other tasks...

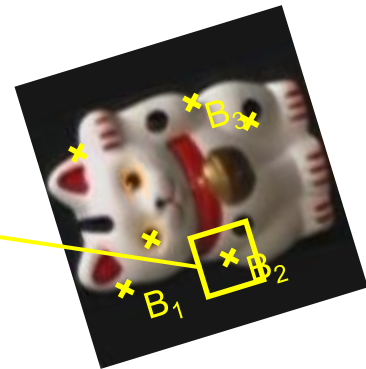
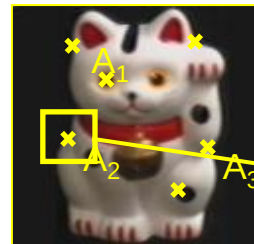
Example: Object recognition via keypoint matching



$$d(f_A, f_B) < T$$

1. Find a set of distinctive keypoints
2. Define a region around each keypoint
3. Extract and normalize the region content
4. Compute a local descriptor from the normalized region
5. Match local descriptors

Key trade-offs



Detection of interest points



More Repeatable

Robust detection
Precise localization

More Points

Robust to occlusion
Works with less texture

Description of patches



More Distinctive

Minimize wrong matches

More Flexible

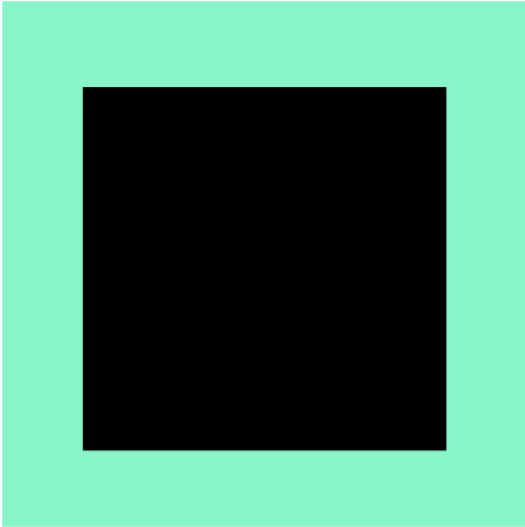
Robust to expected
variations
Maximize correct matches

What are good features?

- Repeatable/precise
 - ✓ The location is “exact”
- Salient/“matchable”
 - ✓ Its description is distinctive (as unique as possible)
- Compact/efficient
 - ✓ $\#Features \ll \#Pixels$
- Local
 - ✓ Uses only a very small neighbourhood around the point -> robustness to occlusion

Finding ‘interest’ points/keypoints

“Interest” point: a point that will likely make a good feature



Could you find interest points in this image?



Where would you tell your friend to meet you?

Finding 'interest' points/keypoints

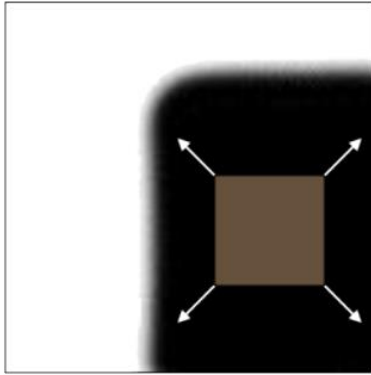
They could be

- ✓ Corners
- ✓ Edges/Lines
- ✓ Blobs
- ✓ etc

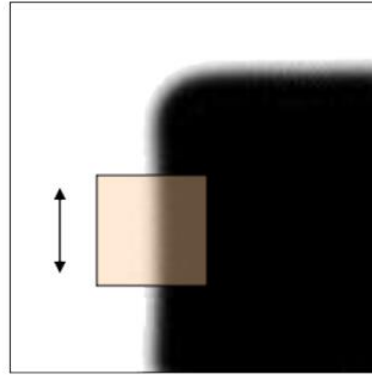


Corner detection

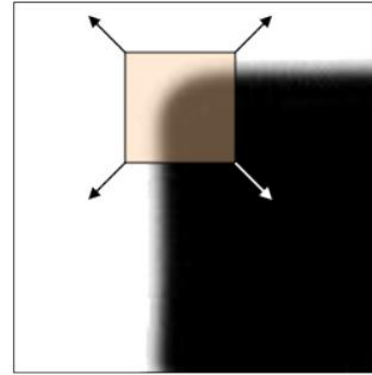
- We should easily recognize the point by looking through a small window
- Shifting a window in *any direction* should give *a large change* in intensity



“flat” region:
no change in
all directions



“edge”:
no change
along the edge
direction



“corner”:
significant
change in all
directions

Corner detectors

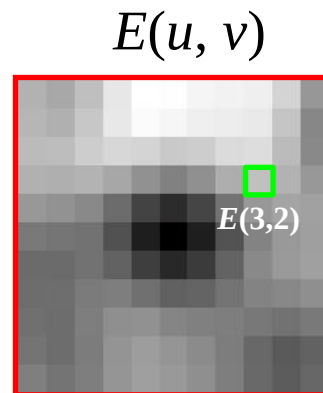
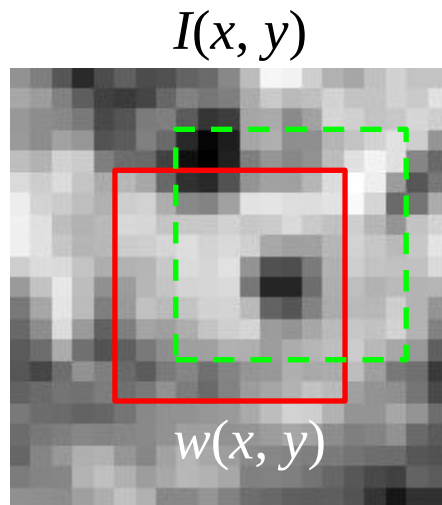
- Proposed by: Harris and Stephens, 1988
- Known as Harris corner detector
- How much does it change for the shift (u, v) :

$$E(u, v) = \sum_{x,y} \underbrace{w(x, y)}_{\text{window function}} \underbrace{[I(x + u, y + v) - I(x, y)]}_{\text{shifted intensity}} \underbrace{I(x, y)}_{\text{intensity}}^2$$

Corner Detection: Compute

Change in appearance of window $w(x,y)$
for the shift $[u,v]$:

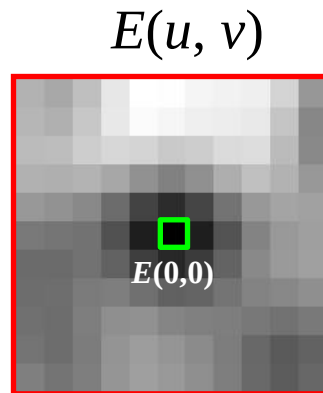
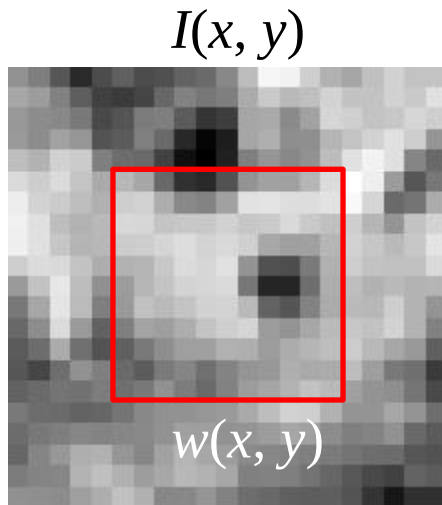
$$E(u, v) = \sum_{x,y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$



Corner Detection: Mathematics

Change in appearance of window $w(x,y)$
for the shift $[u,v]$:

$$E(u, v) = \sum_{x,y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$



Corner Detection: Compute

Change in appearance of window $w(x,y)$
for the shift $[u,v]$:

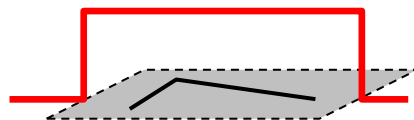
$$E(u,v) = \sum_{x,y} w(x,y) [I(x+u, y+v) - I(x,y)]^2$$

Window
function

Shifted
intensity

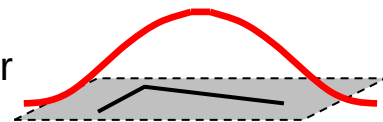
Intensity

Window function $w(x,y) =$



1 in window, 0 outside

or



Gaussian

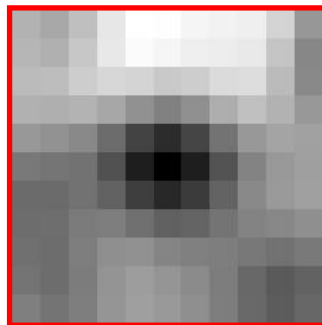
Corner Detection: Compute

Change in appearance of window $w(x,y)$
for the shift $[u,v]$:

$$E(u, v) = \sum_{x,y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

We want to find out how this function behaves for small shifts

$E(u, v)$



Corner Detection: Compute

Change in appearance of window $w(x,y)$
for the shift $[u,v]$:

$$E(u,v) = \sum_{x,y} w(x,y) [I(x+u, y+v) - I(x,y)]^2$$

We want to find out how this function behaves for small shifts

Local quadratic approximation of $E(u,v)$ in the neighborhood of $(0,0)$ is given by the second-order Taylor expansion:

$$E(u,v) \approx E(0,0) + [u \ v] \begin{bmatrix} E_u(0,0) \\ E_v(0,0) \end{bmatrix} + \frac{1}{2} [u \ v] \begin{bmatrix} E_{uu}(0,0) & E_{uv}(0,0) \\ E_{uv}(0,0) & E_{vv}(0,0) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

Corner Detection: Mathematics

The quadratic approximation simplifies to

$$E(u, v) \approx [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

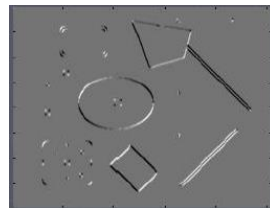
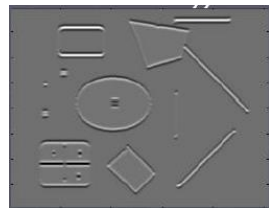
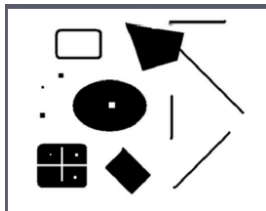
where M is a *second moment matrix* computed from image derivatives:

$$M = \sum_{x,y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Corners as distinctive interest points

$$M = \sum w(x, y) \begin{bmatrix} I_x I_x & I_x I_y \\ I_x I_y & I_y I_y \end{bmatrix}$$

2 x 2 matrix of image derivatives (averaged in neighborhood of a point).



Notation:

$$I_x \Leftrightarrow \frac{\partial I}{\partial x}$$

$$I_y \Leftrightarrow \frac{\partial I}{\partial y}$$

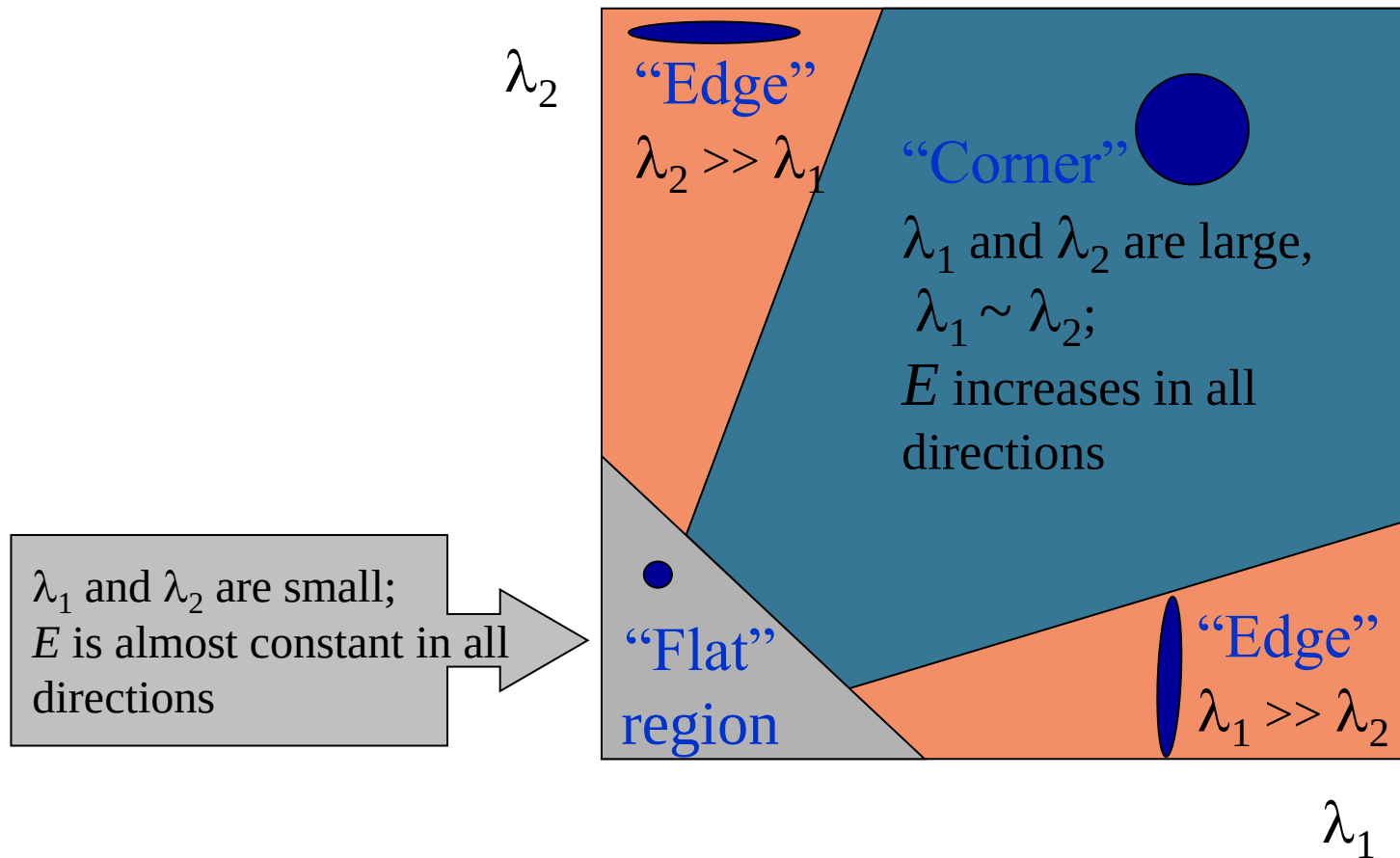
$$I_x I_y \Leftrightarrow \frac{\partial I}{\partial x} \frac{\partial I}{\partial y}$$

Corner Detection

- Simplifying by EIGEN values and EIGEN vectors gives:

$$M \approx \sum_{x,y} \omega(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

Corner Detection



Corner Detection

- Define a corner score (Cornersness)

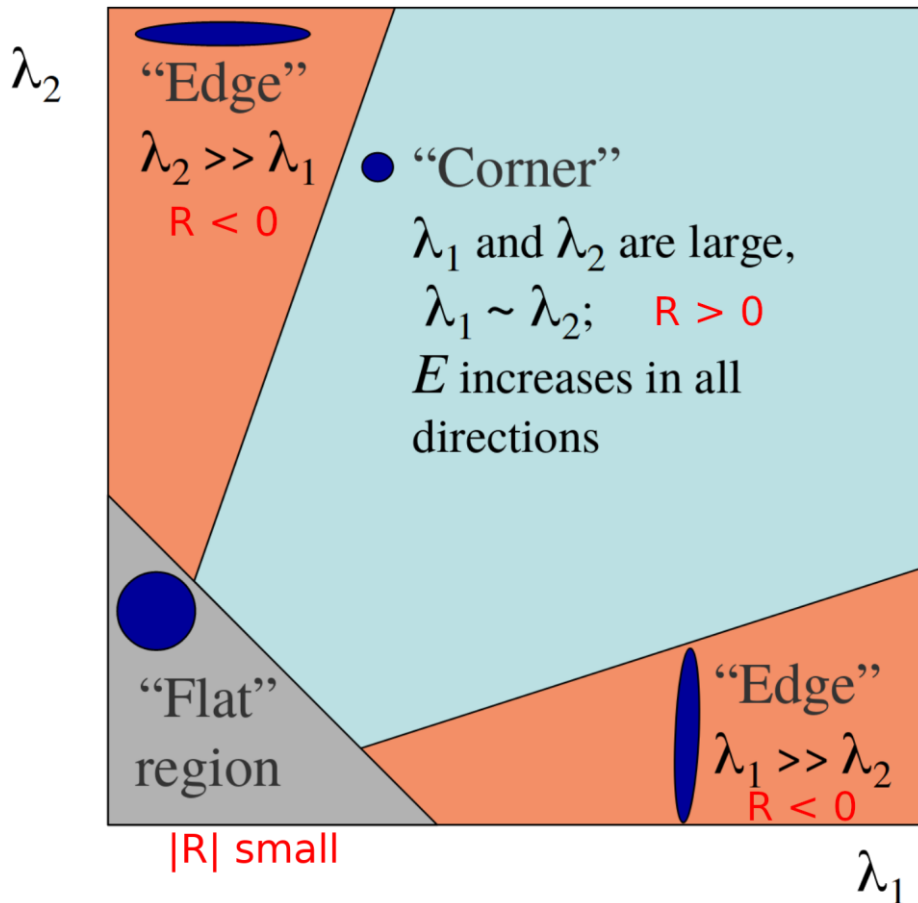
$$R = \det(M) - k(\text{trace}(M))^2$$


$\lambda_1 \lambda_2$ $\lambda_1 + \lambda_2$

- $0.04 < k < 0.06$

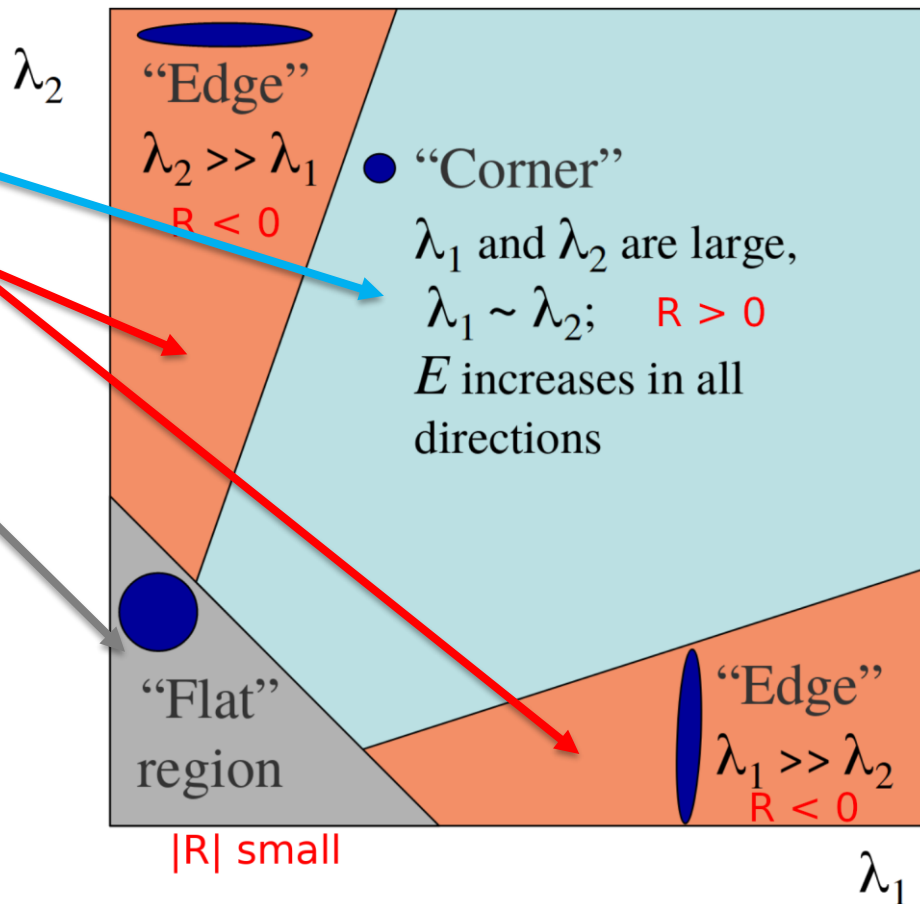
Corner Detection

- Where are the corners?
- Where are the edge region?
- What about constants?



Corner Detection

- Where are the corners?
- Where are the edge region?
- What about constants?



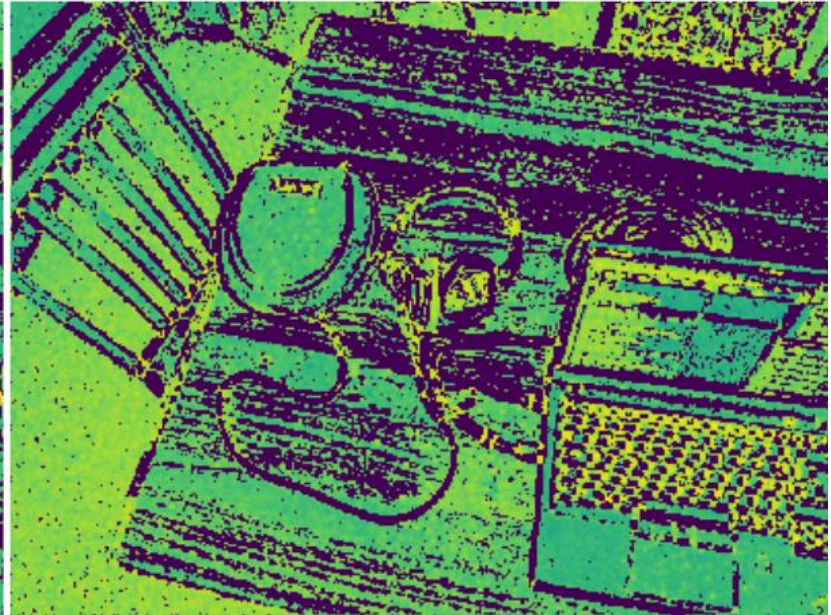
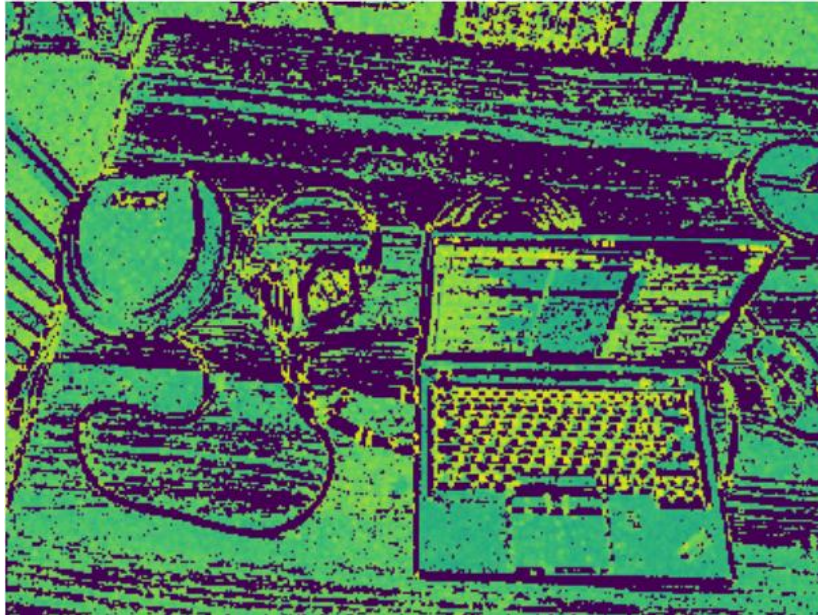
Corner Detection: Algorithm

- Compute Gaussian derivatives of the image
- Compute the autocorrelation matrix M on a Gaussian window around each pixel
- Compute the “corneriness” R for each pixel
- Threshold R
- Apply non-maximum suppression

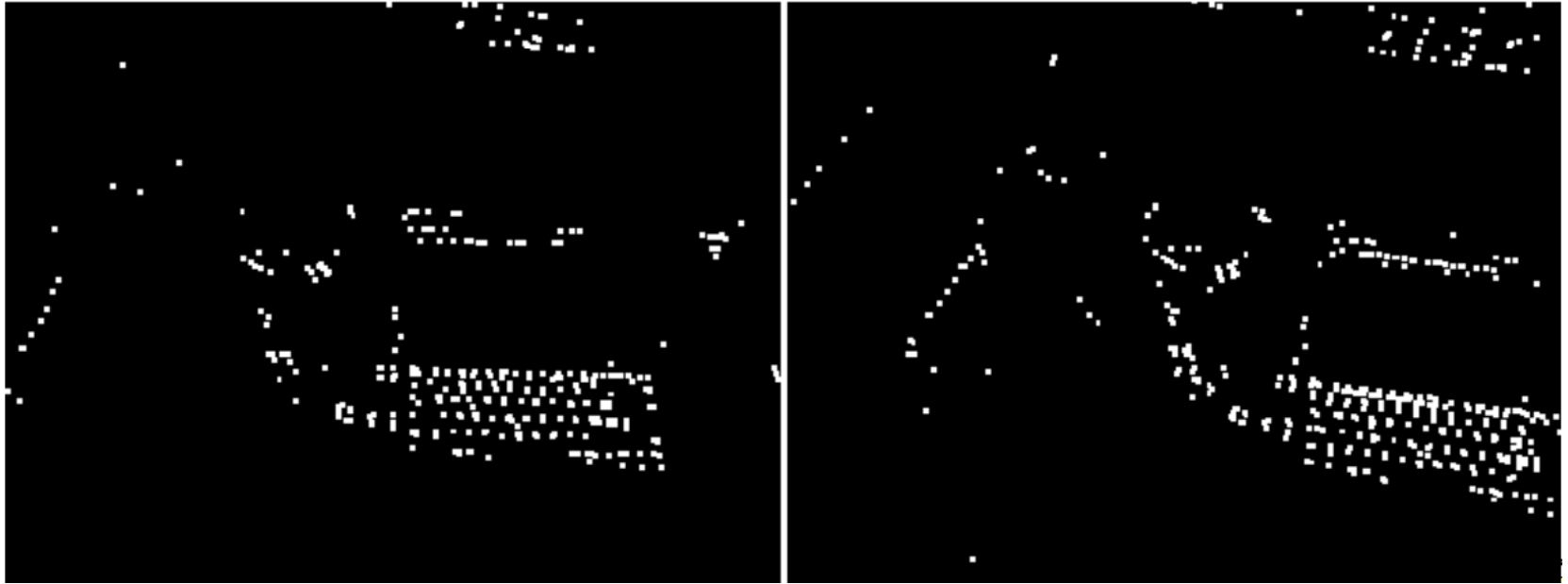
Example



Example: Corner response (R)



Example: $R > T$



Example: Corners



Other approaches to corner detection

- Shi-Tomasi (1994): $\min(\lambda_1, \lambda_2)$
- Triggs (2004): $\lambda_1 - \alpha\lambda_2$
- Brown, Szeliski, Winder (2005): $\frac{\det(A)}{\text{Tr}(A)}$

Properties of Harris detector

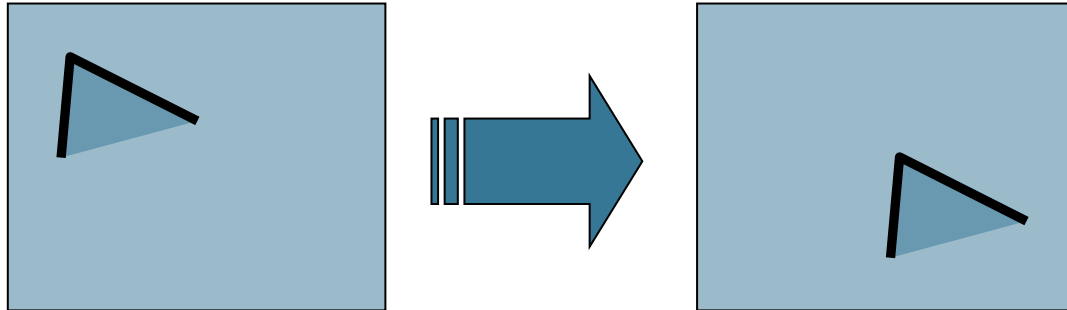
- invariant to translation?
- Invariant to rotation?
- Invariant to intensity change?
- What about invariance to **scale**?

Properties of Harris detector

- invariant to translation? YES, because ...

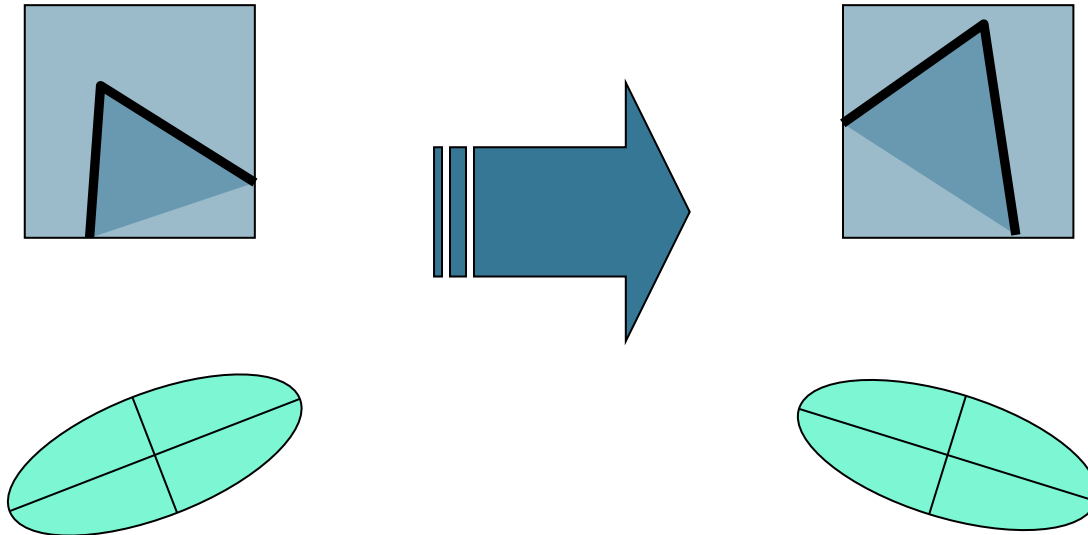
$$E(u, v) = \sum_{x,y} \underbrace{w(x, y)}_{\text{window function}} \underbrace{[I(x + u, y + v) - I(x, y)]}_{\text{shifted intensity}}^2 \underbrace{I(x, y)}_{\text{intensity}}$$

An arrow points from the text "YES, because ..." to the summation symbol $\sum_{x,y}$ in the equation.



Properties of Harris detector

- Obviously, invariant to translation
- Invariant to rotation? **YES, because of eigenvalues**



Properties of Harris detector

- Obviously, invariant to translation
- Invariant to rotation (eigenvalues don't change w.r.t. rotation)
- Invariant to intensity change? **Yep, because ...**

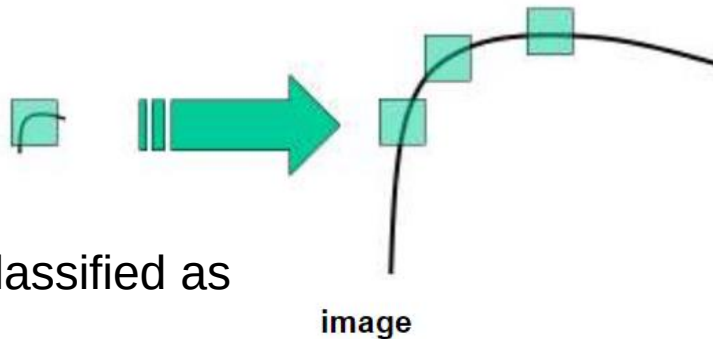
$$E(u, v) = \sum_{x,y} \underbrace{w(x, y)}_{\text{window function}} \underbrace{[I(x + u, y + v)]}_{\text{shifted intensity}} - \underbrace{I(x, y)}_{\text{intensity}}]^2$$


Properties of Harris detector

- Obviously, invariant to translation (why?)
- Invariant to rotation (eigenvalues don't change w.r.t. rotation)
- Invariant to intensity change (i.e. $\hat{I} = \alpha I + \beta$)
- What about invariance to **scale**?

Corner location is not covariant to scaling!

All points will be classified as
edges

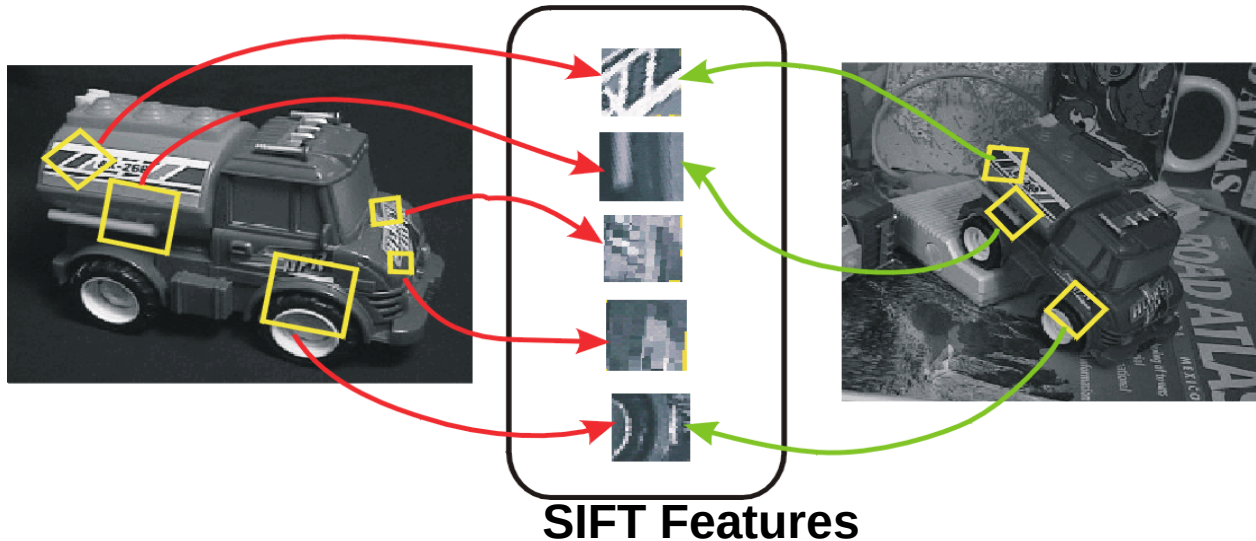


SIFT (Scale-Invariant Feature Transform): Motivation

- The Harris operator is not invariant to scale
- For better image matching, Lowe's goal was to develop an interest operator that is invariant to scale and rotation.
- Also, Lowe aimed to create a **descriptor** that was robust to the variations corresponding to typical viewing conditions. **The descriptor is the most-used part of SIFT.**

Idea of SIFT

- Image content is transformed into local feature coordinates that are invariant to translation, rotation, scale, and other imaging parameters



Quiz

- Which are characteristics of a good feature?
 - Detectable under geometric and intensity changes
 - Possesses distinctive properties
 - Looks pretty
 - Uses a large quantity of information around it
- Regarding the "corneriness" measure produced by the Harris corner detector, which indicates that there is NO corner:
 - $R > 0$ and $|R|$ is large
 - $R > 0$ and $|R|$ is small
 - $R < 0$ and $|R|$ is large

Overall Procedure at a High Level

1. Scale-space extrema detection

Search over multiple scales and image locations.

2. Keypoint localization

Fit a model to determine location and scale.

Select keypoints based on a measure of stability.

3. Orientation assignment

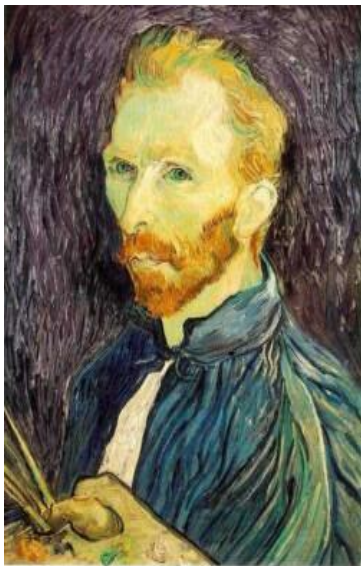
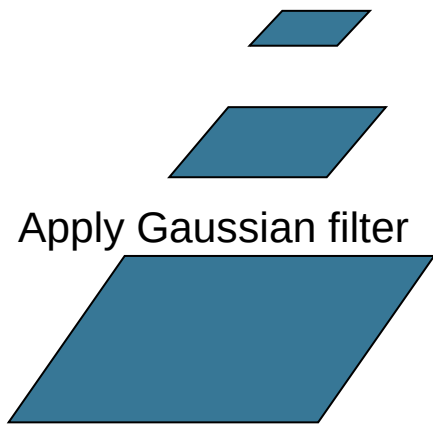
Compute best orientation(s) for each keypoint region.

4. Keypoint description

Use local image gradients at selected scale and rotation to describe each keypoint region.

Scale-space extrema detection

- **Goal:** Identify locations and scales that can be repeatedly assigned under different views of the same scene or object.
- **Method:** search for stable features across multiple scales using a continuous function of scale.



Gaussian 1/2



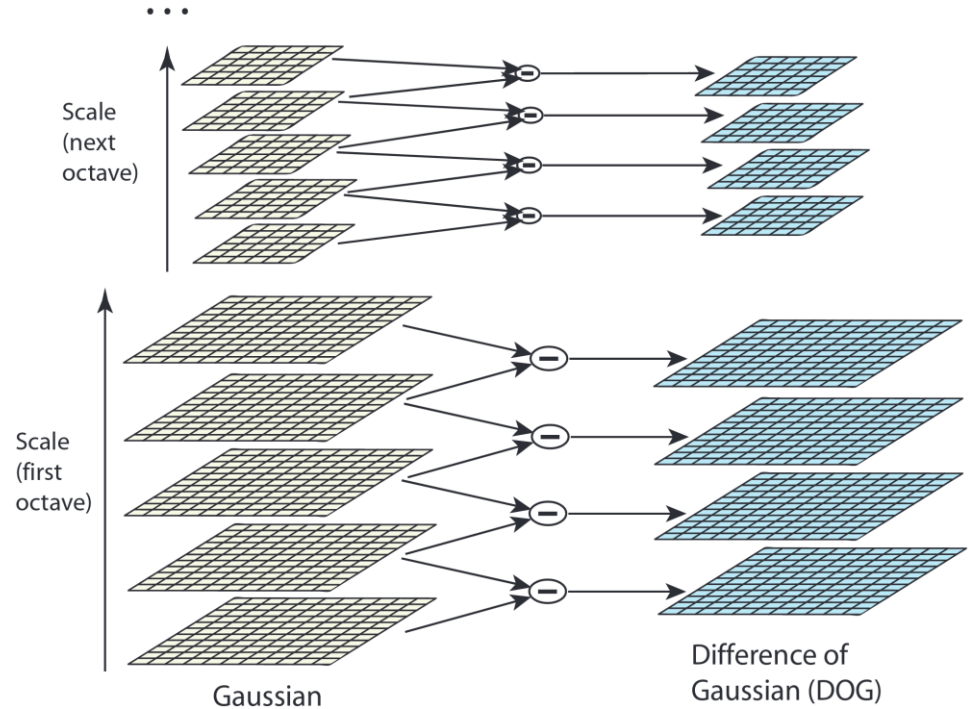
G 1/4



G 1/8

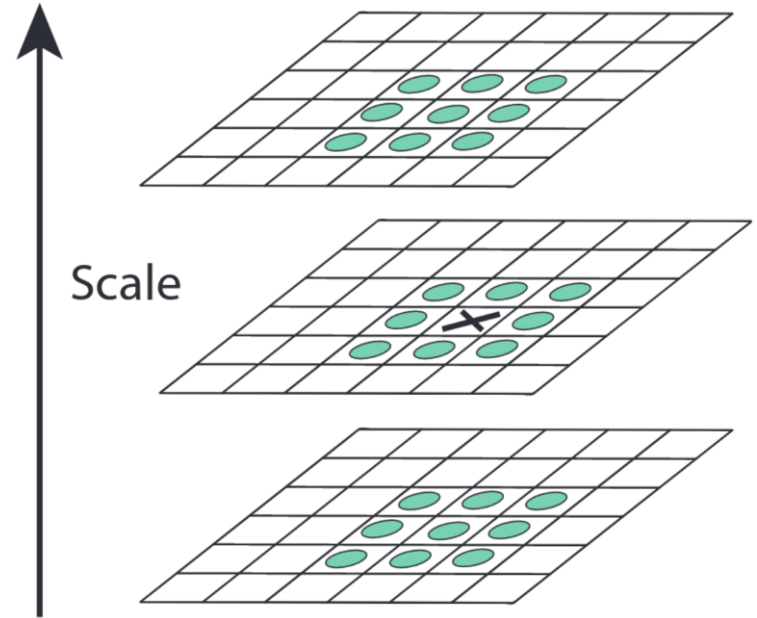
Scale Invariant Feature Transform (SIFT)

- Lowe (2004),
proposed using DoG
filters as
approximation to LoG
for interest point
detection

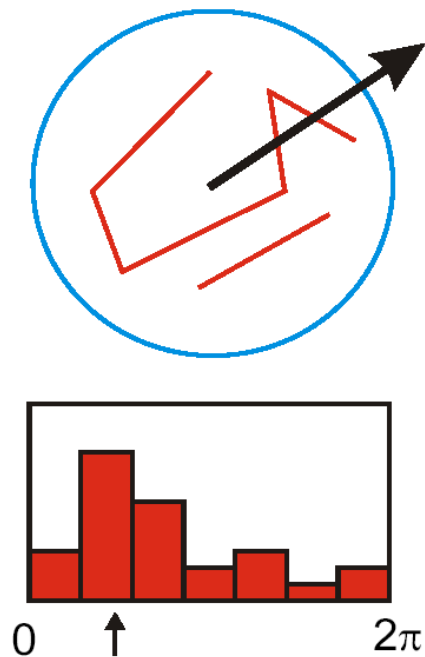


Scale Invariant Feature Transform (SIFT)

- Detect maxima and minima of difference-of-Gaussian in scale space
- Each point is compared to its 8 neighbors in the current image and 9 neighbors each in the scales above and below



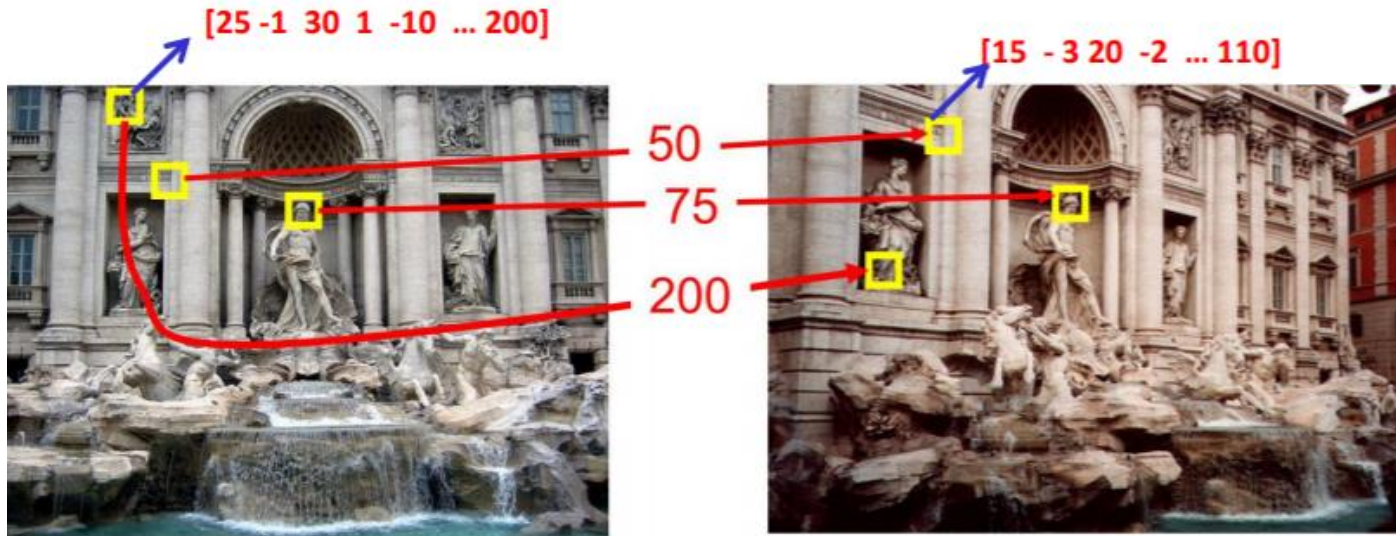
Orientation assignment



- Create histogram of local gradient directions at selected scale
- Assign canonical orientation at peak of smoothed histogram

What now?

- We know how to **detect** points of interest
- We need to find **correspondences** (match them)
- That requires **describing** each point of interest



Descriptor

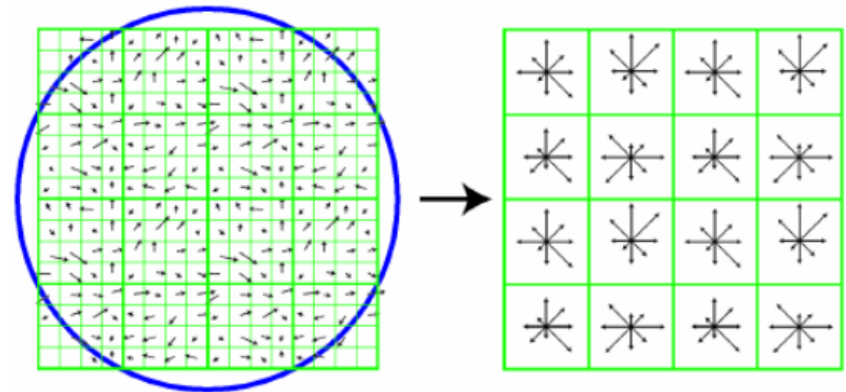
- A descriptor “explains” what goes on around a neighbourhood
Must be distinctive (i.e. different corresponding points can’t have the same description)
- Must be robust to geometric transformations (i.e. different viewpoint)
- Examples of naive descriptors:
 - Histogram of a patch around keypoint
 - Average of a patch around keypoint

Would a naive descriptor work?



SIFT descriptor

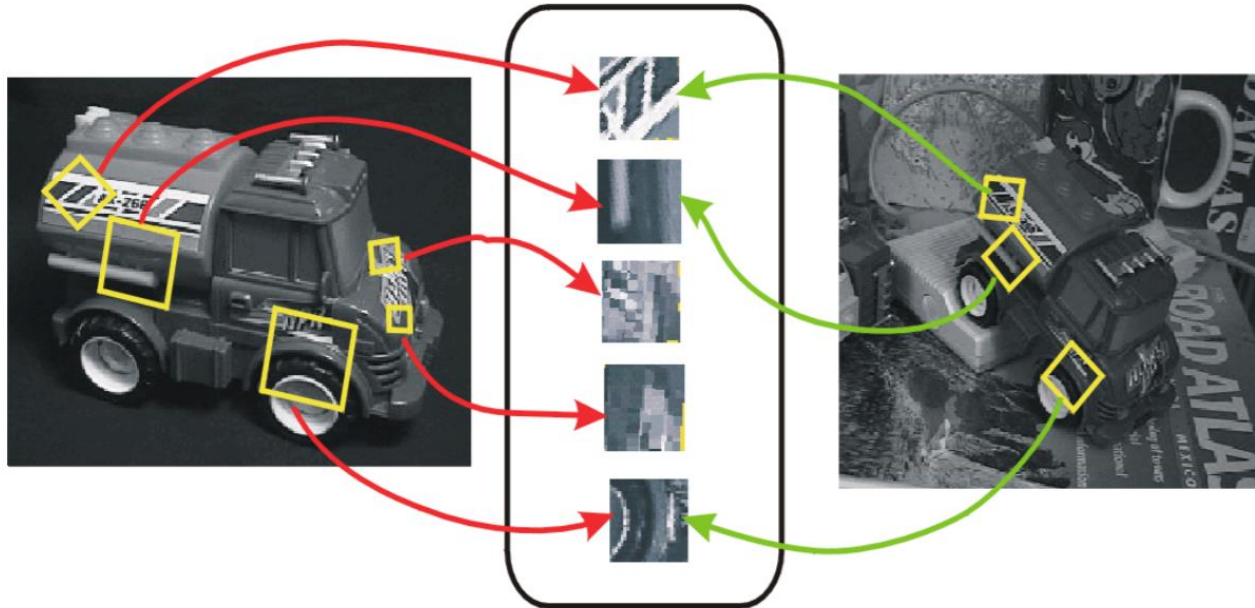
- SIFT calculates the local direction and set it as reference:
 - ▶ Using a 36 bin smoothed histogram, selects the most voted direction over the window
 - ▶ “Rotates” the local area according to that direction
- Generates a vector with 16 histograms of the 8 directions in each 16^{th} of the Gaussian smoothed window:



Matching?

- We can **detect** interest points
- We can **describe** interest points
- How do we match the points detected in different images?
 - ▶ We need to decide which point pairs should be compared
 - ▶ We need to define which matches will be used

Feature Matching

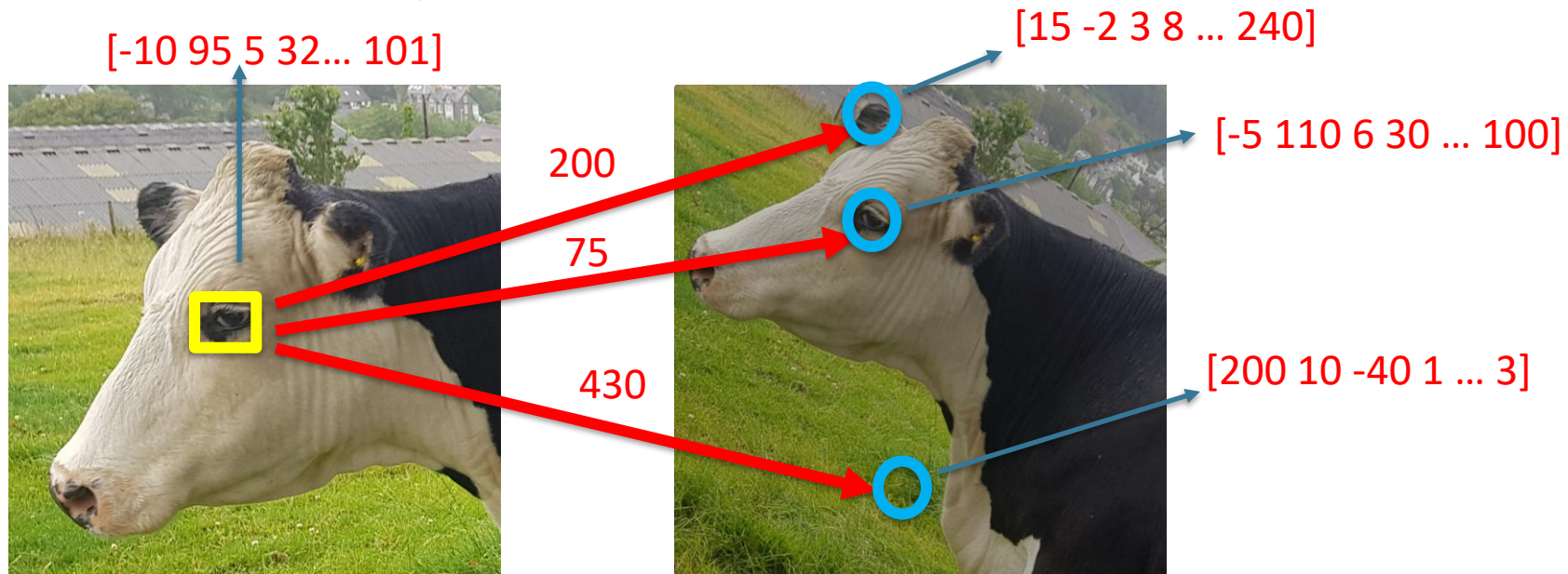


Feature Descriptors

Matching strategy (NNDR=nearest neighbor distance ratio)

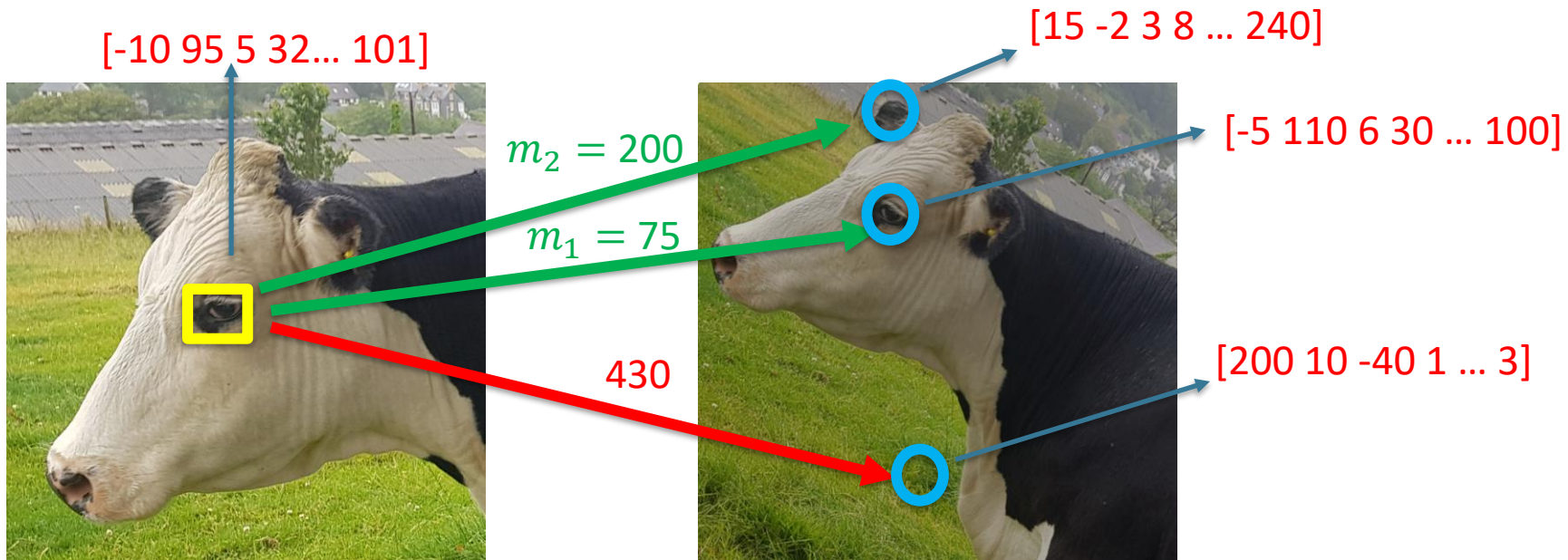
1- Compare all descriptors in one image to all descriptors in another image

Metric: SSD (sum of square differences), NCC, etc



Matching strategy (NNDR)

2- Select the two closest corresponding description vectors to the query one (m_1 = the best match, m_2 = the second-best match)



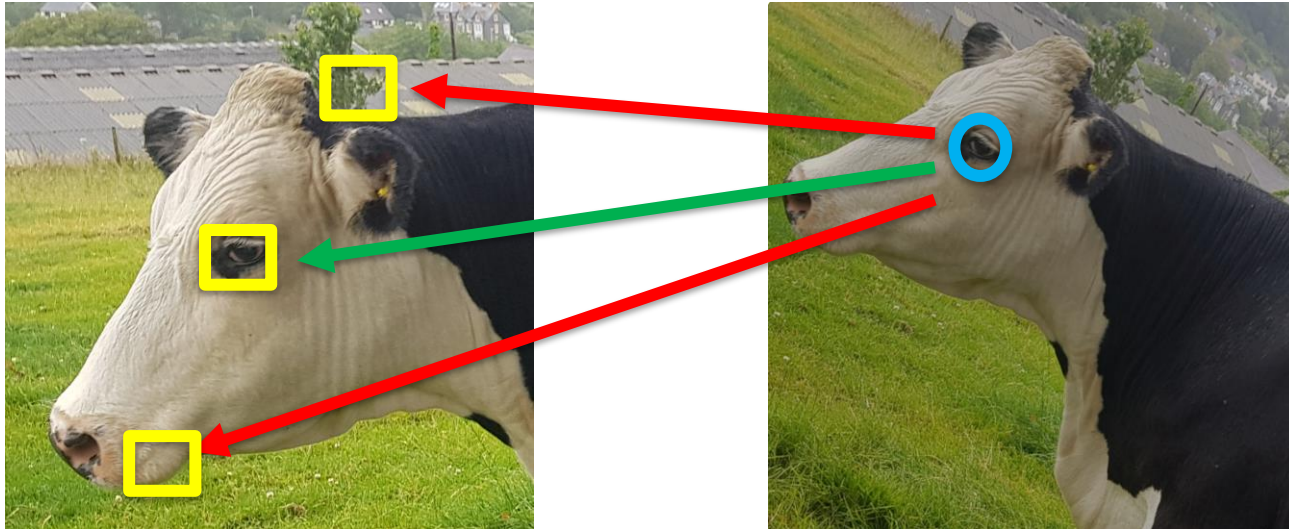
Matching strategy (NNDR)

3- Reject if best match m_1 is not much better than the 2_{nd} best m_2

if $\frac{m_1}{m_2} > T$ (e.g. 0.7) $\rightarrow$ it is *NOT* a good match

Matching strategy (NNDR)

4- Reject if a match from left to right is **not** the same from right to left



Summary

- Detection: to find interest points
- Description: to describe detected points
- Matching: to match interest points using their descriptors

References

- Szeliski, 4.1 – 4.1.3; 6.1 – 6.1.4
- <https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf>